

LAB # 02

Implementation and Analysis of Perceptron

Lab Tasks:-

1. Go to Kaggle and download a binary classification dataset. train a perceptron to predict the target, and test it on new data points.

Code:-

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=100, n_features=2,
                           n_redundant=0, n_clusters_per_class=1)

def step(z):
    return 1 if z > 0 else 0

def perceptron(X, y, lr=0.1, epochs=1000):

    X = np.insert(X, 0, 1, axis=1) # bias
    weights = np.ones(X.shape[1])

    for _ in range(epochs):
        for i in range(X.shape[0]):
            y_hat = step(np.dot(X[i], weights))
            weights = weights + lr * (y[i] - y_hat) * X[i]

    return weights[0], weights[1:]

bias, weights = perceptron(X, y)

print("Weights:", weights)
print("Bias:", bias)

new_point = np.array([0.5, -1])
z = bias + np.dot(new_point, weights)
print("Prediction:", step(z))
```

Output:-

```
Weights: [0.08681561 0.18174076]
Bias: 0.100000000000000014
Prediction: 0
```

2. Modify the perceptron to train on shuffled data multiple times and plot how the decision boundary changes. Compare results for different learning rates (e.g., 0.01, 0.1) and observe which learning rate gives faster convergence.

Code:-

```
def perceptron_shuffle(X, y, lr=0.1, epochs=50):

    X = np.insert(X, 0, 1, axis=1)
    weights = np.ones(X.shape[1])

    for _ in range(epochs):

        # Shuffle data
        indices = np.random.permutation(len(X))
        X = X[indices]
        y = y[indices]

        for i in range(len(X)):
            y_hat = step(np.dot(X[i], weights))
            weights = weights + lr * (y[i] - y_hat) * X[i]

    return weights
```

```

w1 = perceptron_shuffle(X, y, lr=0.01)
w2 = perceptron_shuffle(X, y, lr=0.1)

def plot_boundary(X, y, weights, title):
    bias = weights[0]
    w = weights[1:]

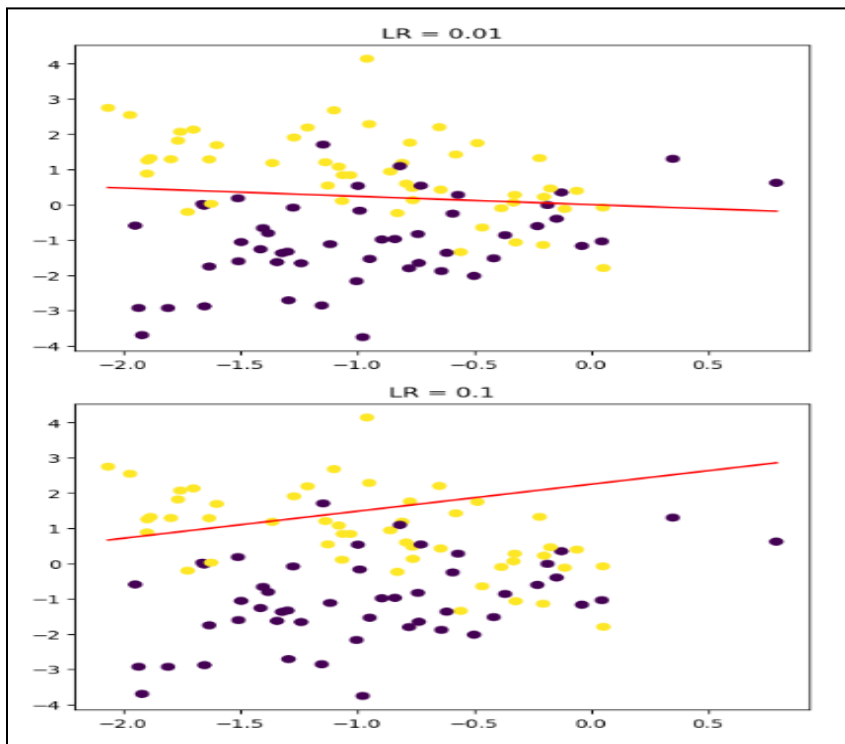
    m = -(w[0] / w[1])
    b = -(bias / w[1])

    x_line = np.linspace(X[:,0].min(), X[:,0].max(), 100)
    y_line = m * x_line + b

    plt.scatter(X[:,0], X[:,1], c=y)
    plt.plot(x_line, y_line, 'r')
    plt.title(title)
    plt.show()

plot_boundary(X, y, w1, "LR = 0.01")
plot_boundary(X, y, w2, "LR = 0.1")

```

Output:-**Bonus Task:**

Take a dataset with 3 features (inputs) and 3 classes from Kaggle or any CSV file. Implement a multi-class perceptron using the one-vs-rest approach. Train the model, make predictions for new samples, and visualize the data in 3D. Analyze which features influence the classification the most and discuss any limitations of the perceptron on this dataset.

Code:-

```

from sklearn.datasets import make_blobs
from mpl_toolkits.mplot3d import Axes3D

X, y = make_blobs(n_samples=150, centers=3, n_features=3, random_state=42)

class MultiPerceptron:

    def __init__(self, lr=0.1, epochs=100):
        self.lr = lr
        self.epochs = epochs

    def train_binary(self, X, y):
        X = np.insert(X, 0, 1, axis=1)
        weights = np.ones(X.shape[1])

        for _ in range(self.epochs):
            for i in range(len(X)):
                y_hat = step(np.dot(X[i], weights))
                weights += self.lr * (y[i] - y_hat) * X[i]

        return weights

    def fit(self, X, y):
        self.classes = np.unique(y)
        self.models = {}

        for c in self.classes:
            y_binary = np.where(y == c, 1, 0)
            self.models[c] = self.train_binary(X, y_binary)

    def predict(self, X):
        predictions = []

        for x in X:
            scores = {}
            for c in self.classes:
                w = self.models[c]
                x_new = np.insert(x, 0, 1)
                scores[c] = np.dot(x_new, w)

            predictions.append(max(scores, key=scores.get))

        return np.array(predictions)

model = MultiPerceptron()
model.fit(X, y)

preds = model.predict(X)

print("Predictions:", preds[:10])

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.scatter(X[:,0], X[:,1], X[:,2], c=y)

ax.set_xlabel("Feature 1")
ax.set_ylabel("Feature 2")
ax.set_zlabel("Feature 3")

plt.title("3-Class Data Visualization")
plt.show()

```

Output:-